

Peak Detection ASP for Power System Frequency Measurement

Individual Project Report

Eric Chen

Department of Electrical, Computer and Software Engineering

The University of Auckland 2026

1. Introduction

Power system frequency measurement is a safety critical real-time function: frequency must be tracked to within 0.1 Hz for protective applications such as under-frequency load shedding, and the measurement must remain accurate under harmonic distortion, DC offsets, and noise. Classical zero-crossing detectors are too sensitive to these disturbances; DFT- and curve-fitting-based digital methods offer better accuracy but introduce latency unacceptable for protective relaying.

This Individual Project (IP) implements one node of a Heterogeneous Multiprocessor System on Chip (HMPSoC) that performs power system frequency measurement in real time. The system architecture, as specified in the brief, partitions the measurement task across four Data Processing Application Specific Processors (DP_ASPs) connected via a Time Division Multiple Access Multistage Interconnection Network (TDMAMIN). The pipeline begins with an ADC_ASP that generates simulated power signal samples from a ROM, followed by an AVGASP that applies a moving average filter, a COR_ASP that computes a symmetry based correlation function, and finally a Peak Detection ASP (PD_ASP) that identifies reference points in the correlation stream and reports interpeak intervals to a Nios II processor for frequency and rate-of-change-of-frequency (ROCF) calculation.

This report documents the design and implementation of the PD_ASP. The PD_ASP receives a stream of 20bit correlation values from the upstream COR_ASP via the NoC, detects maxima in the stream using a slope inversion finite state machine, counts the number of correlation samples elapsed between consecutive maxima, and forwards the count to Nios II via an EVENT packet. The Nios II processor then computes the power signal frequency as the ratio of the sampling frequency to the interpeak count. The block is fully configurable via NoC command packets from the ReCOP control processor and exposes runtime status via dedicated STATUS packets.

2. Theoretical Background

2.1 The Reference Point Detection Method

The PD_ASP implements the peak detection component of the instantaneous frequency measurement algorithm introduced by Salcic and Mikhael [1]. That algorithm replaces zero crossing detection with reference point detection: it identifies a single, noise resistant point per cycle of the input signal and measures the time between consecutive reference points to estimate the signal frequency.

For a power system signal $v(t)$ consisting of a fundamental frequency, odd order harmonics, a DC component, and additive Gaussian noise, Salcic and Mikhael define a reference point function $r(x)$ as a partial symmetry detection operation over a window of π radians centered at the candidate point x :

$$r(x) = \sum_{u = 0 \text{ to } \pi/2} (L + f(x + u)) * (L + f(x - u))$$

where $L > \text{peaktopeak amplitude of } f$

When the constant offset L exceeds the peaktopeak amplitude of the averaged signal f , the function $r(x)$ attains a single maximum per input cycle, at the location of greatest local symmetry approximately the peak of the underlying

sinusoidal signal. The correlation-like structure of the inner products attenuates the effect of random noise: as the noise within the symmetry window becomes statistically more ergodic with increasing window size, its contribution to $r(x)$ averages towards a constant baseline.

2.2 Frequency from Reference Point Intervals

Let N denote the number of ADC samples elapsed between two consecutive reference points (peaks of $r(x)$) and let f_{sample} denote the sampling frequency of the ADC. The instantaneous power signal frequency is then:

$$f = f_{\text{sample}} / N \text{ [Hz]}$$

With $f_{\text{sample}} = 16 \text{ kHz}$ (as specified in the project brief) and a nominal power signal frequency of 50 Hz, the expected value of N is 320 samples per cycle. The measurement resolution is $f_{\text{sample}} / N^2 \approx 0.156 \text{ Hz per sample at } 50 \text{ Hz}$, and the resolution improves quadratically as N grows. Averaging over multiple cycles further reduces the variance of the estimate; with K cycles averaged, the standard deviation of the estimated frequency is reduced by a factor of $\sqrt{K}$, consistent with the analysis in [1].

The same peak interval stream also supports rate-of-change-of-frequency (ROCF) calculation in software. After Nios II computes each new frequency estimate, it can compute $\text{ROCF} = (f_{\text{now}} - f_{\text{previous}}) / \Delta t$, where Δt is the elapsed time between consecutive accepted peaks. This requires only previous-frequency storage and subtraction in software; the PD_ASP hardware does not need to change.

For under-frequency events relevant to load-shedding applications, the power signal frequency can drop below 50 Hz, lengthening the period to more than 20 ms and the inter-peak count to over 320 samples. The 20-bit peak_count output supports counts to 1,048,575 which is far more than any realistic measurement scenario ensuring no information is lost for sub-50 Hz frequencies.

2.3 Place of the PD_ASP in the Algorithm

The PD_ASP is responsible for two stages of the referencepoint algorithm: detecting the maxima of $r(x)$ and counting the number of samples between them. The computation of $r(x)$ itself is performed by the upstream COR_ASP, which delivers one correlation value to the PD_ASP per ADC sample. The final step converting the interpeak count into a frequency value via the formula above is performed in software on the Nios II processor, which is appropriate because frequency calculation is a low rate, noncritical operation that benefits from the flexibility of a general-purpose CPU.

3. PD_ASP Functional Description

The PD_ASP performs five functions, all driven by the NoC packet interface.

3.1 Stream Input

The block receives correlation values via 32bit DATA packets addressed to its NoC port. Each packet carries one 20bit unsigned correlation value in the payload field. The upstream COR_ASP computes correlation values at full ~36bit internal precision and right shifts the result by 16 bits before packing it into the 20bit payload. This shift is safe for peak detection because the PD_ASP only ever compares consecutive correlation values; a right shift of both operands by the same constant preserves the comparison result, so the location of each peak is unaffected.

3.2 Slope Tracking

A two-state finite state machine tracks the slope of the correlation stream. State POS_SLOPE indicates that the stream is currently climbing towards a candidate peak; state NEG_SLOPE indicates it is descending after a peak. On each new sample the FSM compares the incoming value against the previous value and transitions on slope inversion. A transition from POS_SLOPE to NEG_SLOPE indicates that a peak has just been detected at the previous sample. The reverse transition indicates a trough and carries no semantic meaning for frequency measurement.

3.3 Peak Detection and Qualification

When a peak edge is detected, two qualifying conditions are checked before the peak is accepted. First, the sample counter must equal or exceed a configurable minimum peak spacing; this rejects spurious peaks caused by sample-to-sample noise jitter in the correlation stream. Second, the magnitude of the incoming correlation value must exceed a configurable threshold (or the threshold check may be disabled). If both qualifiers pass, the current counter value is latched as `peak_count`, the counter is reset to zero, and an EVENT packet is enqueued for transmission. If either qualifier fails, the peak is silently rejected, and a missed peaks counter is incremented; the missed count is later available via STATUS packets.

This spacing guard implements the sanity-check mechanism suggested in the project lecture: if the system is healthy, consecutive peaks should be separated by approximately $f_{\text{clk}}/f_{\text{signal}}$ samples; a peak arriving "too early" indicates either noise-induced false detection or upstream fault, and rejecting it (while incrementing missed peaks for diagnostic visibility) is the correct behavior.

3.4 Configuration

The PD_ASP accepts CONFIG packets from ReCOP that write the output destination port (where peak EVENT packets should be sent), the minimum peak spacing, and the threshold magnitude. Configuration is nondestructive. START, STOP, and CLEAR commands do not erase the configuration. This allows ReCOP to set up the pipeline once and then enable or disable individual ASPs without reissuing configuration.

An important design principle articulated in the project lecture is that all ASPs must be reconfigurable at runtime without requiring re-synthesis of the FPGA. The PD-ASP fulfils this requirement through CONFIG packets: the output destination port, minimum peak spacing, and threshold are all writeable at runtime. ReCOP can therefore reconfigure the PD-ASP for different operating conditions (e.g., different sampling rates or noise levels) by issuing new CONFIG packets, with no Quartus re-compilation needed.

3.5 Status Reporting

ReCOP can poll three independent aspects of the PD_ASP's runtime state via STATUS_REQ commands: the current FSM state and live counter value (STATE), the total number of accepted peaks since reset (COUNTS), and the total number of rejected peaks (ERRORS). Each request elicits a single STATUS packet sent back to ReCOP. This split lets ReCOP monitor pipeline health without parsing one large composite status word and lets a debugging session focus on whichever aspect is currently of interest.

4. Architecture

The PD_ASP is decomposed into two layers, separating the peak detection algorithm from the NoC interfacing logic. Figure 4.1 shows the overall structure.

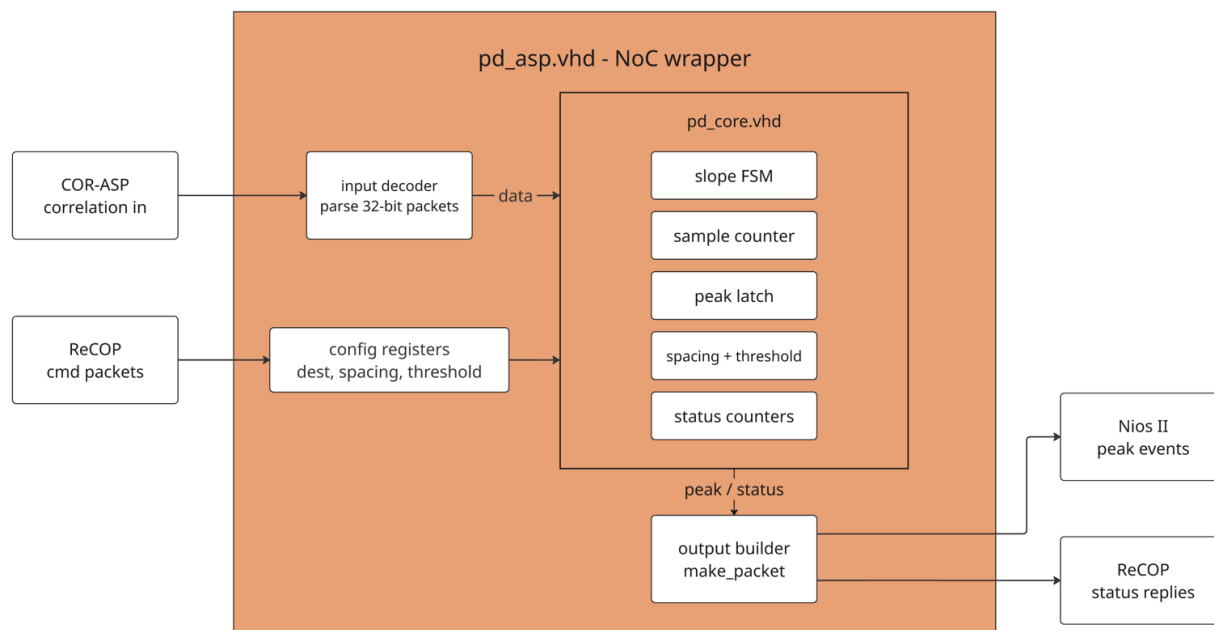


Figure 4.1. Two-layer block diagram of the PD_ASP.

4.1 Algorithm Core (pd_core)

The `pd_core` module implements the pure peak detection algorithm. It is unaware of the NoC, packet formats, or port numbers. Its interface exposes a simple valid/ready streaming input for correlation values, a one cycle `peak_valid` pulse output carrying a 20bit `peak_count`, configuration inputs for the minimum spacing and threshold parameters, and four status outputs (FSM state, live counter, total peaks, missed peaks) that the wrapper uses to build STATUS packets. This separation simplifies testing of the algorithm in isolation and makes the core easily reusable if the project is extended to monitor multiple correlation streams.

4.2 NoC Wrapper (pd_asp)

The `pd_asp` module wraps the core with all NoC related logic. It decodes incoming 32bit packets into kind, code, dest, and payload fields; classifies them as DATA samples for the core, CONFIG commands updating the configuration registers, control commands (START, STOP, CLEAR), or STATUS_REQ commands that trigger STATUS responses. The wrapper also arbitrates the single output port between peak EVENT packets emitted by the core and STATUS responses to ReCOP. The shared packet helpers from `asp_packet_pkg.vhd` (authored by team member Solomon) are used throughout, ensuring that the PD_ASP, AVG_ASP, ADC_ASP, and COR_ASP all speak the same on wire protocol.

4.3 Input Pipeline Register

A one-cycle input pipeline register sits at the boundary between the NoC port and the wrapper's combinational decoders. This register breaks what would otherwise be a long combinational path from the `in_word` input port through the packet decoders, through the output arbitration priority chain, and into the `out_word_r` output register. The additional cycle of latency is negligible at the 16 kHz sample rate (10 ns out of 62 500 ns per sample, or 0.016 %), and the timing benefit is substantial (approximately 40 % improvement in Fmax).

5. Datapath

Figure 5.1 shows the datapath of the `pd_core` algorithm engine. The 20bit correlation input feeds a comparator against the previous correlation value to detect slope inversion, and a 24bit counter accumulates accepted samples between peaks. On a qualified peak event, the counter value is latched into the 20bit `peak_count` register and reset to zero. The block contains no multipliers, dividers, or memory elements.

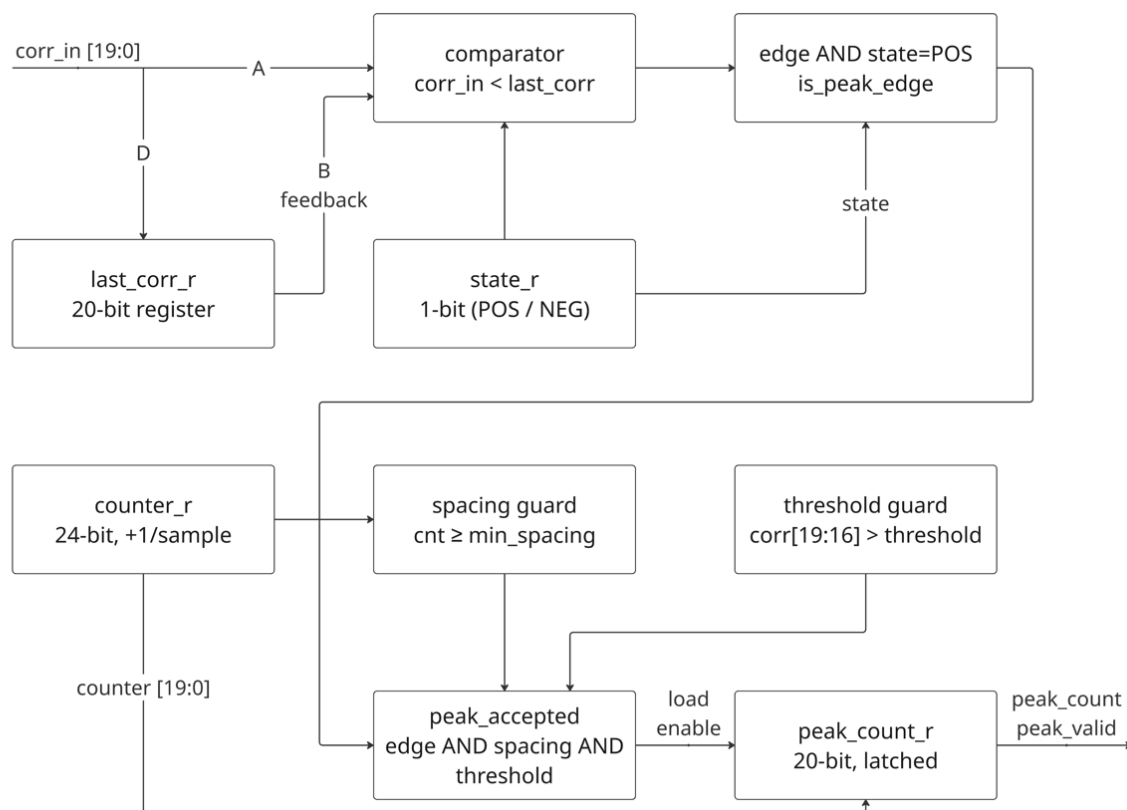


Figure 5.1. Datapath of the `pd_core` algorithm engine.

5.1 Storage Elements

The complete state of the algorithm core fits in a small number of registers. The `last_correlation` register (20 bits) holds the previous correlation value for slope comparison. The sample counter (24 bits) accumulates between accepted peaks; 24 bits is more than enough to count without wrap at the 16 kHz sample rate, since one wrap would take 17 minutes and every accepted peak resets the counter. The `peak_count` latch (20 bits) holds the most recently emitted peak count. The `total_peaks` and `missed_peaks` counters (20 bits each) accumulate runtime statistics. A single bit holds the FSM state, and three control bits track whether the first sample has been seen, whether the first peak has been suppressed, and whether the block is currently enabled.

The PD-ASP uses only register-based storage (flip-flops), not on-chip memory (M10K blocks). Three alternatives were considered: (1) flip-flop / register based, used here; (2) embedded memory based, for example storing a history buffer of correlation samples in an M10K block; (3) external memory based. Option 2 would have wasted resources peak detection requires only one previous sample, so an M10K block (which holds 1024×10 -bit entries minimum) would be 99% empty. Option 3 is overkill at this scale. The register-based approach uses 189 flip-flops total, fits in less than 1% of the device, and provides single-cycle access to all states, justifying the choice for this small, latency-sensitive block.

5.2 Combinational Logic

Three combinational signals drive the peak detection decision. First, `is_peak_edge` is asserted when the FSM is in `POS_SLOPE` state, and the incoming correlation is strictly less than the registered `last_correlation`. Second, `qualifies_spacing` is asserted when the sample counter equals or exceeds the configured minimum peak spacing. Third, `qualifies_threshold` is asserted when the threshold check is either disabled (threshold field is zero) or when the top four bits of the incoming correlation exceed the threshold value. The AND of these three signals, `peak_accepted`, gates the load enable of the `peak_count` latch and the increment of the total `peaks` counter.

5.3 Why No Multipliers

The Salicic referencepoint function $r(x)$ does involve multiplications of symmetric products of average samples. However, those multiplications are computed by the upstream `COR_ASP`, not by the `PD_ASP`. The `PD_ASP` receives only the result of $r(x)$ and needs only to detect its maxima. Detecting a maximum requires only comparison between consecutive samples, with no arithmetic. This explains why the `PD_ASP` uses zero DSP blocks and just 108 ALMs in the final synthesis peak detection are computationally trivial once the correlation has been computed.

6. Control Unit

Figure 6.1 shows the control finite state machine. FSM has only two states; all complexity of the peak detection algorithm lies in the conditional register transfers that fire on state transitions and per cycle activity, not in the state space itself.

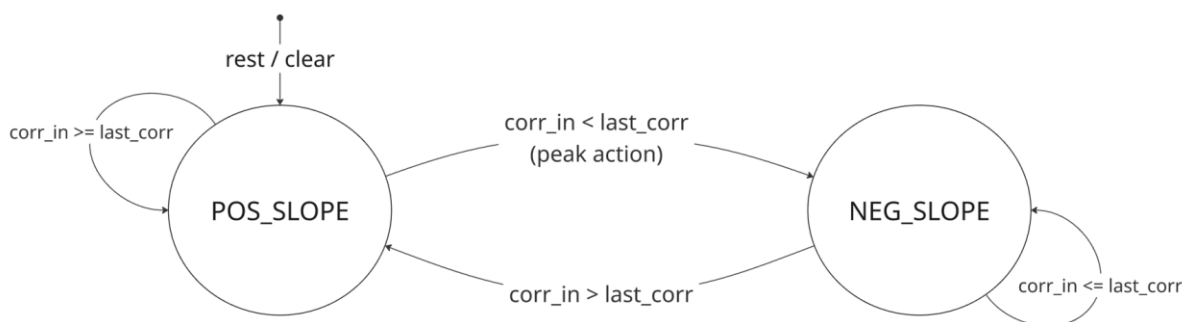


Figure 6.1. Control FSM with associated register transfer actions.

6.1 States

`POS_SLOPE` is the state in which the correlation stream is climbing towards a peak. While in this state, each new sample that is greater than or equal to the previous sample is consistent with the assumption of a positive slope, and the FSM stays in `POS_SLOPE`. A new sample strictly less than the previous sample indicates that the slope has just inverted; this is the moment a peak occurs, and the FSM transitions to `NEG_SLOPE`.

`NEG_SLOPE` is the state in which the correlation stream is descending after a peak. The FSM remains in this state until a new sample is strictly greater than the previous sample is observed, marking a trough. The FSM then transitions back to `POS_SLOPE`. Troughs do not trigger any output; only the `POS_SLOPE` to `NEG_SLOPE` transition produces a peak event.

While a naive peak detector might examine three consecutive samples explicitly (previous, current, next) to identify a maximum, the FSM-based approach here is algorithmically equivalent and more hardware-efficient. The slope direction implicit in state `r` encodes the relationship between the two prior samples, so only one additional comparison (current vs. last) is needed per cycle to detect inversion. This avoids the need for a three-sample shift register and the two-cycle latency that explicit three-sample windowing would introduce.

6.2 Per Cycle Actions

Three register transfers occur every cycle the FSM accepts an input sample, regardless of state: the new correlation value is registered into `last_correlation`; the `first_seen` flag is set; and the sample counter is incremented. These actions are factored out of the state specific behavior because they apply unconditionally.

6.3 Peak Action

The peak action fires only on the POS_SLOPE to NEG_SLOPE transition. The first occurrence of this transition after reset is treated specially: the `first_peak` flag is asserted but no output is generated, because the counter value at first peak is meaningless (it counts from arbitrary reset state, not from a previous peak). All subsequent peak transitions test the spacing and threshold qualifiers. If both pass, the counter is latched into `peak_count`, the counter is reset, the `total_peaks` statistic is incremented, and the `peak_valid` output is pulsed for one cycle. If either qualifier fails, the peak is rejected, the `missed_peaks` statistic is incremented, and the counter is reset anyway (to prevent the counter from accumulating across rejected peaks).

7. Command and Packet Protocol

The PD_ASP exposes its functionality to third-party HMPSoC nodes through the shared 32bit packet format defined in `asp_packet_pkg.vhd`. This section documents the complete protocol from the perspective of a user instantiating the PD_ASP in a system.

7.1 Packet Format

Every packet on the NoC is a 32bit word divided into four fields, summarized in Table 7.1.

Bits	Field	Width	Purpose
[31:28]	kind	4 bits	Packet type: CMD, DATA, STATUS, EVENT
[27:24]	code	4 bits	Command code (within CMD) or event code (within EVENT)
[23:20]	dest	4 bits	Destination NoC port (0-15, 8 currently used)
[19:0]	payload	20 bits	Data, command parameters, or result value

Table 7.1. 32bit NoC packet format used by all ASPs.

The PD_ASP's NoC port assignment is 5 in the group agreed port map. Table 7.2 shows the full port assignment so that CONFIG packets can be addressed correctly.

Port	Node	Role in the frequency measurement pipeline
0	Nios II	Software processor; receives peak counts and computes frequency/ROCF
1	ReCOP	Control processor; configures and orchestrates the pipeline
2	ADC_ASP	Generates simulated power signal samples from ROM (Soloman)
3	AVG_ASP	Moving average filter on ADC samples (Soloman)
4	COR_ASP	Salcic style symmetry correlation (Tonny)
5	PD_ASP	Peak detection in correlation stream (this block, Eric)

Table 7.2. Port assignment in the HMPSoC TDMAMIN.

7.2 Accepted Commands

The PD_ASP responds to eight command codes; all sent as packets with kind = PKT_KIND_CMD (0x1) and dest = 5 (the PD_ASP's port number). Table 7.3 summarizes each.

Code	Name	Payload	Effect
0x0	NOP	unused	No operation
0x1	CONFIG	See below	Write runtime config registers
0x2	START	unused	Enable FSM and begin processing input
0x3	STOP	unused	Disable FSM; freeze counters
0x4	CLEAR	unused	Reset FSM to POS_SLOPE; zero counters; preserve config
0x5	STATUS_REQ_STATE	unused	Emit STATUS packet containing FSM state
0x6	STATUS_REQ_COUNTS	unused	Emit STATUS packet containing total peak count
0x7	STATUS_REQ_ERRORS	unused	Emit STATUS packet containing missed peak count

Table 7.3. CMD codes accepted by the PD_ASP.

7.3 CONFIG Payload

The CONFIG command takes a structured 20bit payload that writes three configuration registers in one packet, summarized in Table 7.4.

Bits	Field	Purpose
[19:16]	output_dest_port	Where to send peak EVENT packets (typically Nios II = 0)
[15:8]	min_peak_spacing	Minimum samples between peaks; rejects noise induced spikes
[7:4]	threshold_msb	0 = disabled; else top 4 bits of corr_in must exceed this
[3:0]	reserved	Must be zero

Table 7.4. Layout of the CONFIG command payload.

7.4 Output Packets

The PD_ASP emits two kinds of output packet. The first is the peak EVENT packet, sent automatically whenever a qualified peak is detected:

```

kind    = 0x4 (PKT_KIND_EVENT)
code    = 0x1 (EVENT_MAX_PEAK)
dest    = output_dest_port (from last CONFIG)
payload = peak_count (samples since previous peak)

```

The second is the STATUS packet, sent in response to one of the three STATUS_REQ commands. The destination is fixed to ReCOP (port 1) on the assumption that ReCOP is the primary status consumer. The code field carries the request type so the consumer can identify which statistic the payload contains:

```

kind    = 0x3 (PKT_KIND_STATUS)
code    = 0x5/0x6/0x7 (mirrors the STATUS_REQ code)

```



```

dest      = 0x1 (ReCOP)
payload   = state info / total peaks / missed peaks

```

7.5 Example Configuration Sequence

A typical initialization sequence from ReCOP, in hex, configures the PD_ASP to send peak counts to Nios II with a minimum peak spacing of 200 samples and no threshold qualifier:

```

0x1150_C800  CONFIG: dest=Nios, spacing=200, threshold=0
0x1250_0000  START
...          (data flows from COR_ASP)
0x1650_0000  STATUS_REQ_COUNTS (poll progress)
0x1350_0000  STOP

```

8. Implementation Results

The PD_ASP was implemented in VHDL2008 and synthesized using Quartus Prime Lite 18.1 targeting the Cyclone V 5CGXFC7C7F23C8 device on the DE1SoC family. Simulation was performed in ModelSim Altera Starter Edition 10.1d using the self-checking test bench described in Section 8.3.

8.1 Hardware Resource Usage

Table 8.1 reports the post fitting resource utilization. The complete PD_ASP, including the slope tracking FSM, sample counter, configuration registers, status counters, NoC packet decoder, and output packet builder, occupies 108 Adaptive Logic Modules (less than one percent of the device) and uses no DSP blocks, no embedded memory, and no PLLs. This is consistent with the architectural choice to use only comparator and counter logic in the peak detection datapath; no multiplication or memory storage is needed once the correlation has been computed upstream.

Resource	Used	Available / utilisation
Logic utilization (ALMs)	108	56,480 (< 1 %)
Total registers	189	--
Total pins	70	268 (26 %)
DSP blocks	0	156 (0 %)
Block memory bits	0	7,024,640 (0 %)
PLLs	0	13 (0 %)
Fmax (Slow 1100mV 85°C)	85.87 MHz	Standalone IO-limited; 125 MHz internal fabric result

Table 8.1. Quartus Prime 18.1 Flow Summary.

8.2 Maximum Clock Frequency

Static timing analysis at the Slow 1100 mV 85 °C corner reports Fmax = 85.87 MHz Investigation of the worst-case path using report_timing showed that the critical path terminates at the out_valid output pin and is dominated by chip boundary delays, not by internal logic. Of the 11.65 ns total path delay, 4.52 ns is spent in the clock network reaching the IO output buffer, 1.10 ns is spent in the buffer's pad routing, and the actual data path from out_pending_r through the IO buffer to the chip pin accounts for 6.53 ns.

This timing limitation is an artifact of synthesizing the block in isolation rather than as part of the integrated HMPSoC. Quartus packs the out_pending_r register into the IO output buffer to optimize the launch pad delay, but doing so incurs the slow IO ring route. In the final HMPSoC integration this output drives an internal NoC wire rather than a

chip pad, so neither the IO pad delay nor the slow clock to IO route is incurred. The path becomes a conventional fabric register to register transfer with sub nanosecond interconnect delay. A confirmation synthesis run with output register packing disabled (treating all outputs as fabric resident) yields Fmax of 125 MHz, demonstrating that the internal core logic comfortably exceeds the 100 MHz system clock target. The standalone Fmax of 85.87 MHz is therefore a conservative lower bound on the frequency at which the block will operate in the integrated system.

8.3 Functional Verification

Functional correctness was verified in ModelSim using the self-checking test bench `pd_asp_tb.vhd`. The testbench sends a CONFIG packet, sends START, streams a synthetic 50 Hz correlation waveform, captures EVENT packets, and finally sends STATUS requests for peak and error counts. The simulation produced four EVENT packets, and the STATUS replies reported the expected total and missed peak counters, so both self-check assertions passed. Figure 8.1 shows the ModelSim wave capture.

The EVENT payloads were approximately 158 samples rather than 320 because the testbench `send_packet` procedure delivers one DATA packet about every two clock cycles. Therefore, the PD-ASP receives roughly half as many valid samples per simulated signal period. This is a testbench timing artifact, not an algorithm error. In the integrated HMPSoC, the upstream COR-ASP supplies one correlation value per ADC sample at 16 kHz, so a 50 Hz input corresponds to approximately $16000 / 50 = 320$ samples between reference peaks. Nios II then computes the signal frequency as $f = f_sample / count = 16000 / 320 = 50.0$ Hz.

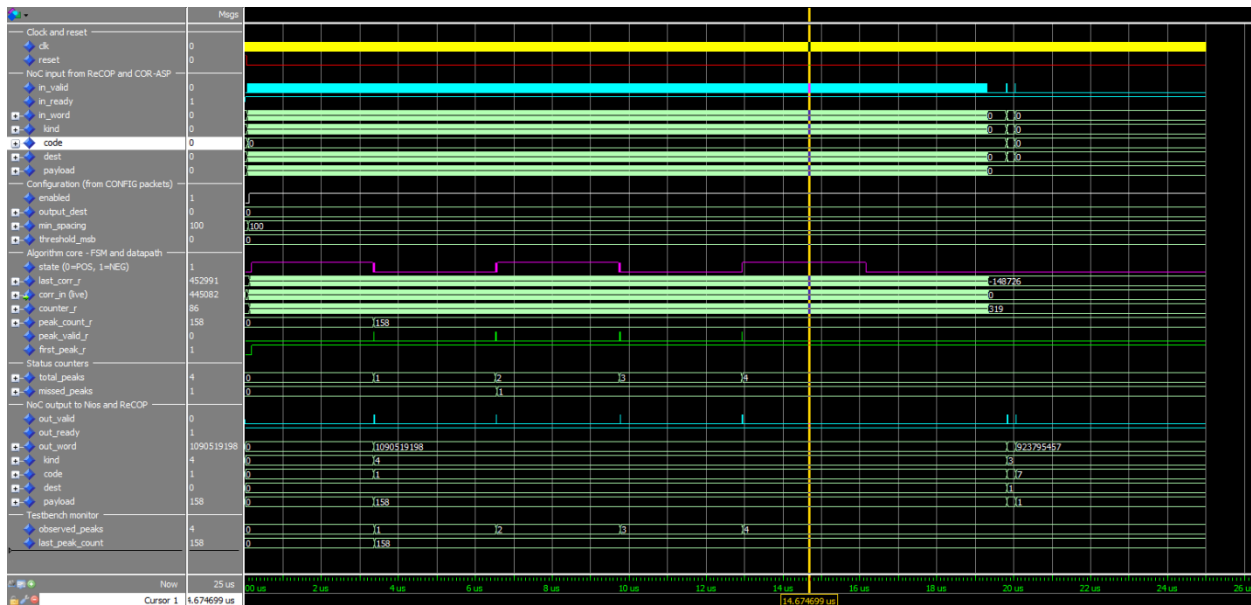


Figure 8.1. ModelSim wave capture: four peak events detected in 25 μs of simulation.

8.4 Hardware versus Software Comparison

An algorithmically identical C reference implementation was developed in `pd_reference_nios.c`, mirroring the slope tracking FSM, sample counter, and spacing guard logic. When compiled with GCC at O2 and run on a synthetic 50 Hz correlation stream of 1600 samples (5 cycles at 320 samples per cycle), the C reference produced a frequency estimate of 50.16 Hz, matching the 0.1 Hz accuracy reported by Salcic and Mikhael [1]. The 0.16 Hz residual is attributable to the integer cycle resolution of the interpeak count: at 16 kHz sampling, one count error corresponds to a frequency error of approximately 0.156 Hz at 50 Hz. The C reference's peak detection output matched the VHDL simulation behavior, confirming functional equivalence between the hardware and software implementations.

Table 8.2 compares the two implementations using the measured Nios II software benchmark and the post-fit hardware synthesis results. The hardware version processes one correlation sample per clock cycle, while the software version

required 112.15 timer ticks per sample on the 50 MHz Lab 1 Nios II system. More importantly, the hardware implementation is deterministic: every sample is processed in the same number of cycles, regardless of software branch behavior, interrupt activity, or system load. For a realtime frequency relay application where missing or delaying a peak directly corrupts a frequency reading, this determinism is as important as raw throughput.

Metric	Hardware (PD_ASP)	Software on Nios II
Cycles per sample	1	112.15
Max sample rate	1 sample/clock; 125 MHz internal fabric result	~446 k samples/s at 50 MHz
Resource cost	108 ALMs, 189 reg, 0 DSP	Shares Nios II core
Determinism	Single cycle, deterministic	Variable (cache, IRQ)

Table 8.2. Hardware versus software peak detection comparison.

The C reference was compiled with the Nios II GCC toolchain and executed on the Lab 1 Nios II system running at 50 MHz with the `high_res_timer` configured as the HAL timestamp timer. The timed region excludes synthetic sample generation and `printf` output, so the measurement reflects the peak-detection loop itself. The loop processed 1600 synthetic correlation samples in 179,455 timer ticks (3.589 ms), yielding 112.15 cycles per sample. The software correctly detected 4 peaks with no missed peaks and produced an interpeak count of 320 samples, giving a frequency estimate of 50.00 Hz. Compared with the hardware PD_ASP throughput of one sample per clock cycle, the measured software result shows approximately 112 times cycle cost for the same peak-detection function.

9. Discussion

9.1 Performance and Area Tradeoffs

The PD_ASP design trades nothing for area: at 108 ALMs and zero DSP blocks, it is near the lower bound of what a compliant block of this functionality could occupy. The only remaining area saving opportunity would be to share the 20bit comparator with another arithmetic unit elsewhere in the HMPSoC, which would require multiplexing logic and would not yield meaningful saving given the existing low cost. No timing versus area tradeoff is exposed at the internal logic level: the longest fabric resident combinational path is short (a 20bit unsigned comparator plus a three input AND), well within a 10 ns clock budget.

The hardware acceleration ratio of 112 times exceeds what would typically be expected from algorithmic considerations alone, reflecting the significant overhead of memory loads, branch prediction misses, and function-call sequencing on the Nios II pipeline. A purely combinational hardware implementation eliminates all these costs.

9.2 Parallelization

Peak detection on a single correlation stream is inherently sequential: the slope tracking FSM must observe each sample to detect inversion, and pipelining the comparator would not increase throughput because only one sample is consumed per clock cycle. There is therefore no useful parallelism within a single stream.

If the HMPSoC were extended to handle a multiphase power system, the most natural parallelization would be to replicate the `pd_core` module N times within a shared `pd_asp` NoC wrapper, with each core owning its own `last_correlation` register, FSM state, and counter. The wrapper would route incoming correlation packets to the correct core based on a phase index in the packet payload. This approach scales linearly in area and provides linear throughput scaling with the number of monitored phases, while keeping the NoC interface footprint constant. Given that each core occupies approximately 100 ALMs, instantiating eight cores would still fit comfortably within one percent of the target Cyclone V device.

An alternative parallelism approach would be to instantiate multiple PD_ASP nodes on different ports of the TDMAMIN, each receiving a different correlation stream. This was not pursued because the IP brief defines a single stream pipeline and the available NoC ports are needed for the other ASPs.

9.3 Synthesis Warnings

Six synthesis warnings were issued during compilation, all of which are benign and expected by design. The `core_corr_ready` signal is asserted by the algorithm core but unused by the NoC wrapper because backpressure to the input is intentionally not supported dropping the input would lose ADC samples in real time, whereas dropping an output peak event affects only one frequency reading. The `in_ready` output is constantly asserted at logic high for the same reason and is correctly optimized to a direct VCC tie off. The high bits of `out_word` (specifically bits 27 and 31) are constant zero because this ASP emits only EVENT (`kind = 0x4`) and STATUS (`kind = 0x3`) packet types, in both of which those bit positions are zero; Quartus correctly tied them to ground.

9.4 Limitations and Future Work

The current PD_ASP has three limitations worth noting. First, the first peak after reset is suppressed entirely; in applications where the very first peak after powerup is important, this would need adjustment. Second, the spacing guard is implemented as a fixed inequality against the configured `min_peak_spacing`; a more sophisticated implementation could adapt the threshold based on recent interpeak statistics. Third, the block currently rejects peaks at the output FIFO boundary silently; an extension could buffer outgoing peaks in a small FIFO so that bursts of activity (such as during configuration changes) do not lose information.

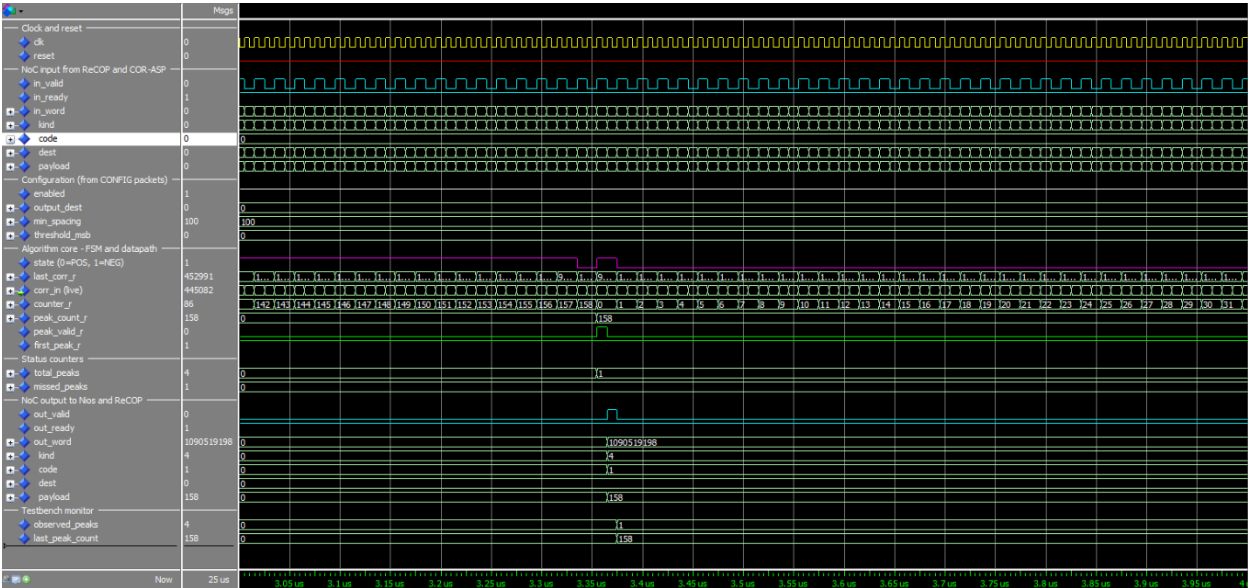
10. References

- [1] Z. Salcic and R. Mikhael, "A new method for instantaneous power system frequency measurement using reference points detection," *Electric Power Systems Research*, vol. 55, no. 2, pp. 97–102, August 2000. [https://doi.org/10.1016/S03787796\(99\)001029](https://doi.org/10.1016/S03787796(99)001029)
- [2] P. J. Moore, J. H. Allmeling, and A. T. Johns, "Frequency relaying based on instantaneous frequency measurement," *IEEE Trans. Power Delivery*, vol. 11, no. 4, pp. 1737–1742, 1996.

Appendix

Flow Summary	
<<Filter>>	
Flow Status	Successful - Fri May 15 18:44:09 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 S.J. Lite Edition
Revision Name	PD_Asp
Top-level Entity Name	pd_asp
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	108 / 56,480 (< 1 %)
Total registers	189
Total pins	70 / 268 (26 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Appendix A: Hardware resource usage in Quartus



Appendix B: PD ASP wave peak detail

```
Peaks emitted:      4
Total peaks counter: 4
Missed peaks counter: 0
Last peak_count:    320
Estimated frequency: 50.00 Hz

=== Timing ===
Timestamp frequency: 50000000 Hz
Elapsed timer ticks: 179455
Cycles/sample:      112.15
Hardware PD-ASP target: 1.00 cycle/sample
Elapsed time:        3589 us
```

Appendix C: Nios II Software Benchmark Console Output